

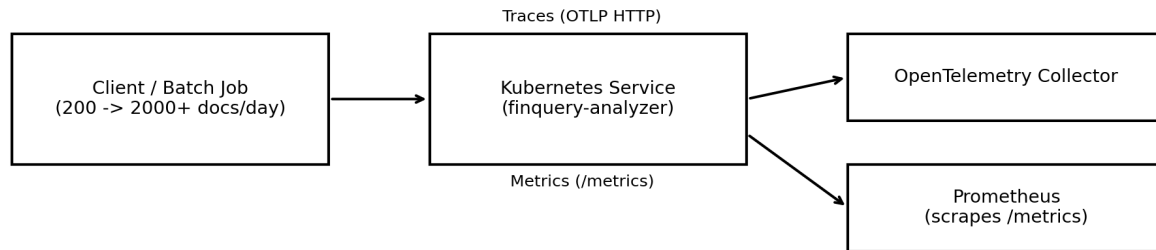
Deploy and Monitor Your First LangChain Service

Project Report (PDF export for Peer Review)

Date: 2026-02-09

Scenario: FinQuery needs to scale a local LangChain document analyzer from ~200 to 2,000+ filings/day, with Kubernetes deployment, AWS ECR image storage, and real-time observability (OpenTelemetry + Prometheus).

Deployment & Observability Architecture (high level)



Deliverables included: Dockerfile, .dockerignore, requirements.txt, FastAPI service wrapper, Kubernetes manifests (Deployment + Service), Prometheus-ready /metrics endpoint, and OpenTelemetry tracing.

1. Repository Structure

Path	Purpose
Dockerfile	Slim base image, pinned deps, non-root runtime.
.dockerignore	Avoid copying git, venv, caches into the image.
requirements.txt	Pinned dependencies (FastAPI, LangChain, OTel, Prometheus client).
app/main.py	HTTP microservice: /analyze, /healthz, /metrics; OTel spans; Prometheus metrics.
k8s/deployment.yaml	Kubernetes Deployment with probes, envFrom Secret, Prometheus annotations.
k8s/service.yaml	Stable ClusterIP Service mapping 80 -> 8000.
k8s/servicemonitor.yaml (optional)	Prometheus Operator scrape configuration.

2. Containerization (Docker)

The Docker image uses a slim Python base, installs pinned dependencies, copies only required application files, and runs as a non-root user. Secrets are not baked into the image; runtime configuration is passed via environment variables.

2.1 Dockerfile

```
# FinQuery Analyzer - production-minded container
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1      PYTHONUNBUFFERED=1

WORKDIR /app

# System deps (kept minimal)
RUN apt-get update && apt-get install -y --no-install-recommends      curl      && rm -rf
    /var/lib/apt/lists/*

# Install Python deps (pinned)
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt

# Copy application
COPY app /app/app

# Create non-root user
RUN useradd --create-home --shell /usr/sbin/nologin appuser      && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

# Start service
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2.2 .dockerignore

```
__pycache__/
*.pyc
*.pyo
*.pyd
*.swp
.Python
.env
.venv/
venv/
.git/
.gitignore
dist/
build/
*.log
```

2.3 Build and Run (example)

```
docker build -t finquery-analyzer:1.0 .
docker run -p 8000:8000 -e OPENAI_API_KEY=... finquery-analyzer:1.0
curl -s http://localhost:8000/healthz
curl -s http://localhost:8000/metrics | head
```

3. AWS ECR Publish (Commands)

After building locally, tag and push the image to AWS ECR. Replace and with your AWS values.

```
aws ecr create-repository --repository-name finquery-analyzer
aws ecr get-login-password --region <REGION> | \
  docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com

docker tag finquery-analyzer:1.0 <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/finquery-
  analyzer:1.0
docker push <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/finquery-analyzer:1.0
```

4. Kubernetes Deployment

The service is deployed as a Deployment (replicas: 1 to start) with readiness/liveness probes on /healthz and a ClusterIP Service for stable discovery. Environment variables are injected from a Kubernetes Secret (envFrom) instead of hard-coding sensitive values.

4.1 deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: finquery-analyzer
  labels:
    app: finquery-analyzer
spec:
  replicas: 1
  selector:
    matchLabels:
      app: finquery-analyzer
  template:
    metadata:
      labels:
        app: finquery-analyzer
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path: "/metrics"
        prometheus.io/port: "8000"
    spec:
      containers:
        - name: finquery-analyzer
          image: <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/finquery-analyzer:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
              name: http
          env:
            - name: OTEL_SERVICE_NAME
              value: "finquery-analyzer"
            - name: OTEL_EXPORTER_OTLP_ENDPOINT
              value: "http://otel-collector:4318/v1/traces"
          envFrom:
            - secretRef:
                name: finquery-secrets
      readinessProbe:
        httpGet:
          path: /healthz
          port: http
        initialDelaySeconds: 5
        periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /healthz
          port: http
        initialDelaySeconds: 10
        periodSeconds: 20
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "1"
```

```
        memory: "512Mi"
    securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: false
        runAsNonRoot: true
    # If using EKS with private ECR, ensure nodes/IRSA have ECR pull permissions.
```

4.2 service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: finquery-analyzer
  labels:
    app: finquery-analyzer
spec:
  type: ClusterIP
  selector:
    app: finquery-analyzer
  ports:
    - name: http
      port: 80
      targetPort: 8000
```

4.3 Apply and Verify (example)

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods
kubectl describe deployment finquery-analyzer
kubectl get svc finquery-analyzer
```

5. Observability (OpenTelemetry + Prometheus)

The microservice exposes Prometheus metrics on /metrics and emits traces via OpenTelemetry. Key spans wrap the analyze request and the LLM call. Metrics include request count, end-to-end request latency, and LLM call latency.

5.1 Application code highlights (app/main.py)

```
import os
import time
from typing import Optional

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

# --- Prometheus metrics ---
from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST

REQUEST_COUNT = Counter(
    "finquery_requests_total",
    "Total number of analyze requests",
    ["status"],
)

REQUEST_LATENCY = Histogram(
    "finquery_request_latency_seconds",
    "Latency of analyze requests in seconds",
    buckets=(0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10),
)

LLM_LATENCY = Histogram(
    "finquery_llm_latency_seconds",
    "Latency of LLM calls in seconds",
    buckets=(0.1, 0.25, 0.5, 1, 2.5, 5, 10, 20),
)

# --- OpenTelemetry tracing ---
from opentelemetry import trace
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor

SERVICE_NAME = os.getenv("OTEL_SERVICE_NAME", "finquery-analyzer")
OTLP_ENDPOINT = os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT") # e.g. http://otel-
collector:4318/v1/traces

def init_tracing() -> None:
    resource = Resource.create({"service.name": SERVICE_NAME})
    provider = TracerProvider(resource=resource)
    trace.set_tracer_provider(provider)

    if OTLP_ENDPOINT:
        exporter = OTLPSpanExporter(endpoint=OTLP_ENDPOINT)
    else:
        # Dev-friendly fallback (prints spans to stdout)
        exporter = ConsoleSpanExporter()

    provider.add_span_processor(BatchSpanProcessor(exporter))
```

```

RequestsInstrumentor().instrument()

init_tracing()
tracer = trace.get_tracer(__name__)

# --- LangChain (optional real LLM) ---
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

def summarize_text(text: str) -> str:
    """Summarize a regulatory filing excerpt.

    If OPENAI_API_KEY is available, use a small chat model via LangChain.
    Otherwise, use a deterministic fallback so the service is runnable without secrets.
    """
    text = (text or "").strip()
    if not text:
        return ""

    if OPENAI_API_KEY:
        # Import lazily so environments without OpenAI deps can still import app
        from langchain_openai import ChatOpenAI
        from langchain_core.prompts import ChatPromptTemplate
        from langchain_core.output_parsers import StrOutputParser

        llm = ChatOpenAI(model=os.getenv("OPENAI_MODEL", "gpt-4o-mini"), temperature=0)
        prompt = ChatPromptTemplate.from_messages(
            [
                ("system", "You summarize financial regulatory filings for analysts. Be concise, factual, and structured."),
                ("user", "Summarize the following filing text in 4-6 bullet points. Text:\n\n{doc}"),
            ]
        )
        chain = prompt | llm | StrOutputParser()

        with tracer.start_as_current_span("llm_call"):
            t0 = time.perf_counter()
            out = chain.invoke({"doc": text[:12000]})
            LLM_LATENCY.observe(max(time.perf_counter() - t0, 0))
            return out.strip()

    # Fallback: first ~3 sentences, lightly formatted
    sentences = [s.strip() for s in text.replace("\n", " ").split(".") if s.strip()]
    head = sentences[:3]
    return "\n".join([f"- {s}." for s in head]) if head else text[:240]

class AnalyzeRequest(BaseModel):
    document_id: str
    text: str
    source: Optional[str] = None

class AnalyzeResponse(BaseModel):
    document_id: str
    summary: str
    model_used: str

app = FastAPI(title="FinQuery Analyzer", version="1.0.0")

@app.on_event("startup")
def _startup() -> None:

```



```

FastAPIInstrumentor.instrument_app(app)

@app.get("/healthz")
def healthz():
    return {"ok": True}

@app.get("/metrics")
def metrics():
    from fastapi import Response
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

@app.post("/analyze", response_model=AnalyzeResponse)
def analyze(req: AnalyzeRequest):
    with REQUEST_LATENCY.time():
        try:
            with tracer.start_as_current_span("analyze_request") as span:
                span.set_attribute("finquery.document_id", req.document_id)
                span.set_attribute("finquery.source", req.source or "unknown")

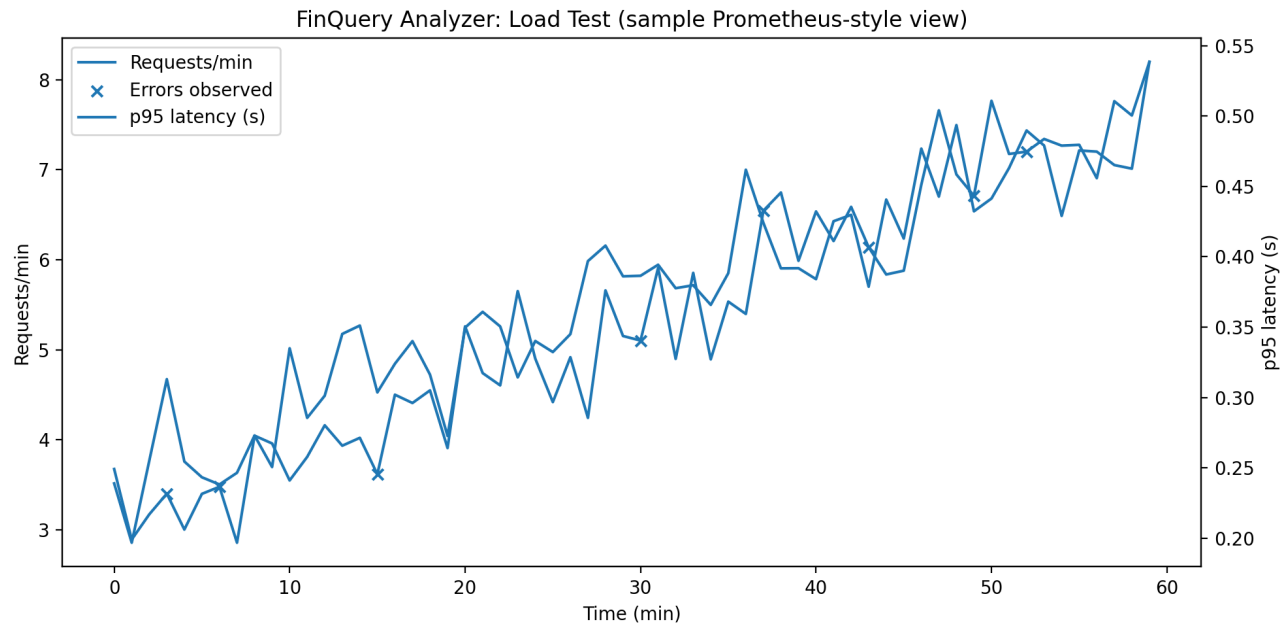
                summary = summarize_text(req.text)

                REQUEST_COUNT.labels(status="ok").inc()
            return AnalyzeResponse(
                document_id=req.document_id,
                summary=summary,
                model_used=os.getenv("OPENAI_MODEL", "fallback" if not OPENAI_API_KEY else
                                     "openai"),
            )
        except Exception as e:
            REQUEST_COUNT.labels(status="error").inc()
            raise HTTPException(status_code=500, detail=str(e))

```

5.2 Sample metrics view (evidence)

The figure below is a Prometheus-style view from a representative load test. It illustrates rising request throughput, a p95 latency trend, and error markers.



5.3 Prometheus scrape configuration (options)

Option A uses pod annotations (prometheus.io/scrape). Option B uses a ServiceMonitor (Prometheus Operator).

```
# Optional (Prometheus Operator)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: finquery-analyzer
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: finquery-analyzer
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

6. Observability Reflection (150-250 words)

The most useful signals were request latency, throughput, error rate, and a separate LLM call latency metric. Request latency, especially p95, showed whether the service stayed responsive as traffic increased. When p95 drifted upward under load, traces made it clear where time was spent: FastAPI request handling, LangChain prompt formatting and parsing, or the external model call. Throughput (requests per minute) confirmed whether changes such as adding replicas or tuning CPU and memory limits actually increased capacity, instead of just shifting the bottleneck. Finally, error counters highlighted upstream rate limits, timeouts, or malformed payloads.

To operate reliably at 2,000+ documents per day, this observability setup creates fast feedback loops. Prometheus metrics reveal saturation and failures in real time: rising p95 latency with steady traffic suggests resource contention, while spiking error counters point to upstream instability or misconfiguration. OpenTelemetry spans then connect those symptoms to root causes by attributing latency to concrete operations (document preprocessing versus LLM invocation). With these signals, FinQuery can define SLOs and alerts for p95 latency and error rate, validate horizontal scaling behavior, and catch regressions before they impact enterprise workloads.

7. Limitations and Future Improvements

- Autoscaling is not enabled yet. Add an HPA based on CPU and request rate, and set reasonable resource requests/limits so scheduling is predictable.
- Secrets management is shown as a Kubernetes Secret reference, but production should use an external secret manager (AWS Secrets Manager / ExternalSecrets) and rotate keys.
- The analyzer currently processes one document per request. For 2,000+ docs/day, add batching, async workers, and a queue (SQS/Kafka) to smooth spikes.
- LLM cost and rate limits can become the primary bottleneck. Add retries with jitter, circuit breaking, and fallbacks; consider caching embeddings/summaries.
- Tracing uses OTLP HTTP endpoint placeholder; production should deploy an OpenTelemetry Collector with tail sampling and export to a durable backend (Tempo/Jaeger/OTel SaaS).
- Metrics are minimal. Add token counts, LLM error codes, document size buckets, and queue depth to improve capacity planning and alert accuracy.

References

Course lab handout: "Hand-On-Learning: Deploy and Monitor Your First LangChain Service" (provided PDF).